

Fully Connected Neural Network

Final Technical Report

NumPy From Scratch | MNIST Feature Interpretation | Deep-FCNN Robustness

3 experimental parts	105 Adult input features	97.37% best MNIST accuracy
--------------------------------	------------------------------------	--------------------------------------

Report scope	This report presents the problem definitions, methodology, architecture choices, hyperparameters, results, visualizations, and interpretations for all three completed FCNN experiments.
---------------------	--

Final submission report

1. Executive Summary

A concise view of the three experiments and their main findings.

The assignment investigates Fully Connected Neural Networks from three complementary perspectives. Part 1 builds an FCNN entirely with NumPy, exposing forward propagation, binary cross-entropy, manual backpropagation, and vanilla SGD. Part 2 uses PyTorch on MNIST to inspect learned first-layer weights and test whether a fully connected model requires the original spatial arrangement of pixels. Part 3 stresses a deep FCNN on a 10-class Tiny ImageNet subset to study vanishing gradients and optimization choices.

76.61% Adult test accuracy	97.27% racy	31.60% best Tiny ImageNet ablation	-19.40 pp largest ablation drop
--------------------------------------	-----------------------	---	---

Key results

Part	Experiment	Result
1	Raw Adult Census	76.61% test accuracy
1	MinMax Adult Census	76.37% test accuracy; lower final loss
2	Normal MNIST	97.27% accuracy
2	Scrambled MNIST	97.37% accuracy
3.1	Deep Sigmoid	Severe first-layer gradient attenuation
3.1	ReLU + BatchNorm	Stronger gradients and continued loss reduction
3.2	Best ablation	No Dropout: 31.60% validation accuracy
3.2	Largest negative impact	Adam -> SGD: -19.40 percentage points

Central finding	FCNNs can learn effective representations from flattened inputs, but they do not have an inherent spatial prior. As depth increases, activation functions, normalization, learning rate, and optimizer choice become increasingly important.
------------------------	--

2. Part 1 - No-Framework Challenge

Binary income classification using a manually implemented FCNN.

2.1 Problem Definition

The objective was to implement a three-layer fully connected neural network for binary income classification on the UCI Adult Census Income dataset without using a neural-network framework. The implementation had to include He initialization, zero biases, ReLU activations, a sigmoid output, binary cross-entropy loss, manual backpropagation, and vanilla SGD.

2.2 Dataset and Preprocessing

The predefined adult.data and adult.test split was used. The dataset contains 32,561 training examples and 16,281 test examples with 14 original input variables. Missing values represented by '?' were converted to missing values and categorical missing entries were filled using the training-set mode. Categorical variables were one-hot encoded, producing 105 input features.

Item	Value
Training set	32,561 rows
Test set	16,281 rows
Original features	14
Features after one-hot encoding	105
Target	$\leq 50K \rightarrow 0$; $> 50K \rightarrow 1$
X_train shape	(32,561, 105)
X_test shape	(16,281, 105)

2.3 Architecture and Methodology

Layer	Configuration
Input	105 features
Hidden 1	64 neurons + ReLU
Hidden 2	32 neurons + ReLU
Output	1 neuron + Sigmoid
Initialization	He initialization; zero biases
Loss	Binary Cross-Entropy
Optimizer	Vanilla SGD
Training	5,000 iterations; learning rate 0.001

The network was implemented with NumPy functions for parameter initialization, ReLU and its derivative, sigmoid, forward propagation, loss computation, backward propagation, parameter updates, and training.

Implementation note

The loss function was corrected for the shape difference between y with shape (n,) and predictions with shape (n,1) by reshaping the target to (-1,1).

2.4 Part 1 Results

Raw features versus MinMax-scaled features under the same architecture and training settings.

Metric	Raw	MinMax scaled
Initial loss	14.0912	0.6931
Final loss	0.565254	0.445844
Training accuracy	76.09%	75.91%
Test accuracy	76.61%	76.37%
Initial dW1	126,771.934	0.304967
Final dW1	0.0	0.075506

76.61%
raw test accuracy

76.37%
scaled test accuracy

0.445844
scaled final loss

Interpretation

MinMax scaling substantially improved optimization behavior and reduced the final loss, but it did not improve classification accuracy in this experiment.

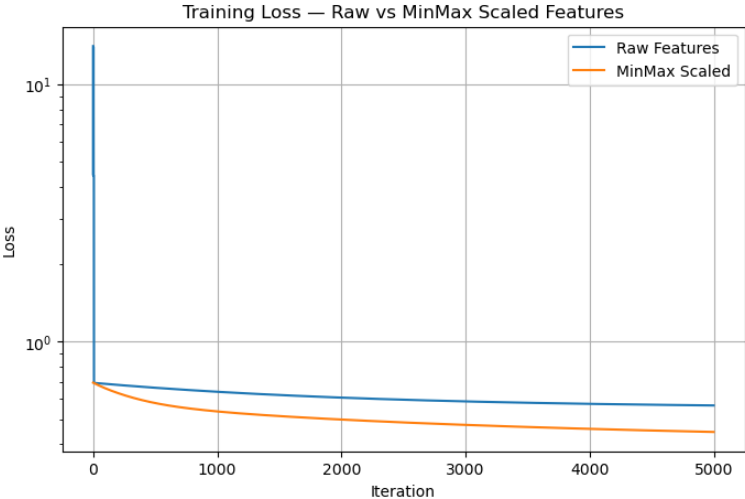


Figure 1. Training loss comparison. The logarithmic y-axis highlights the very large initial raw-feature loss.

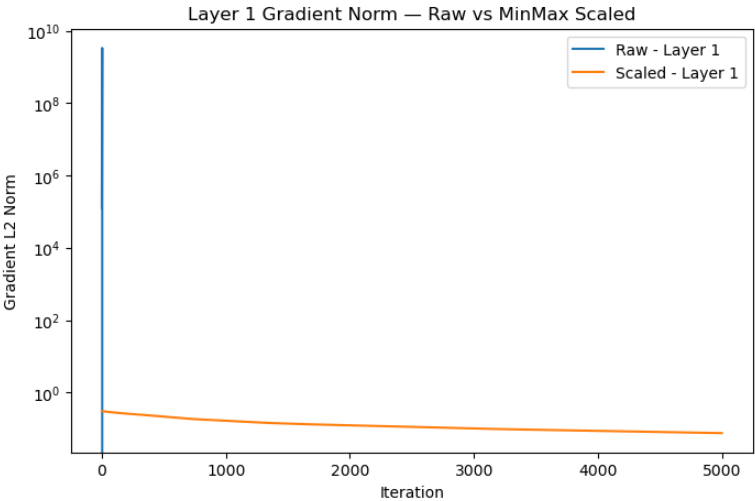


Figure 2. First-layer gradient norm comparison. The logarithmic scale makes the difference in gradient magnitude visible.

2.5 Gradient Analysis

Why feature scaling changed the optimization behavior.

The raw Adult features have very different numerical scales. For example, age and hours-per-week are on relatively small ranges, whereas capital-gain can reach 99,999.

Gradient relationship	$dW1 = (1/m) X^T dZ1$. Large input values can therefore produce very large first-layer gradients.
------------------------------	--

The initial first-layer gradient norm was approximately 126,772 for raw features and 0.305 after MinMax scaling. The raw model's gradients eventually reached zero, whereas the scaled model retained non-zero gradients and reached a lower loss. The correct conclusion is that scaling changed the magnitude and balance of gradients and produced more stable optimization; it should not be described as an accuracy improvement.

Assignment requirement	The raw model achieved 76.61% test accuracy, only about 0.23 pp above the approximately 76.38% majority-class baseline. Because the target is imbalanced, accuracy should not be interpreted as strong predictive performance.
-------------------------------	--

3. Part 2 - MNIST: Vision and Feature Interpretation

Understanding what a flattened FCNN learns from handwritten digits.

3.1 Problem Definition

Part 2 moves to PyTorch and MNIST. The goals are to visualize the first hidden-layer weights as 28 x 28 heatmaps and to determine whether an FCNN depends on the original spatial arrangement of image pixels.

3.2 Dataset and Preprocessing

The MNIST IDX files were loaded directly. There are 60,000 training images and 10,000 test images. Each image is 28 x 28 grayscale with pixel values from 0 to 255. Pixels were normalized to [0,1] and then flattened into 784 input features.

3.3 Architecture and Hyperparameters

Component	Choice
Input	784 flattened pixels
Hidden 1	128 neurons + ReLU
Hidden 2	64 neurons + ReLU
Output	10 logits
Loss	Cross-Entropy
Optimizer	Adam
Learning rate	0.001
Epochs	5

97.27% normal MNIST accuracy	97.37% scrambled MNIST accuracy	5 training epochs
--	---	-----------------------------

3.4 First Hidden-Layer Weight Visualization

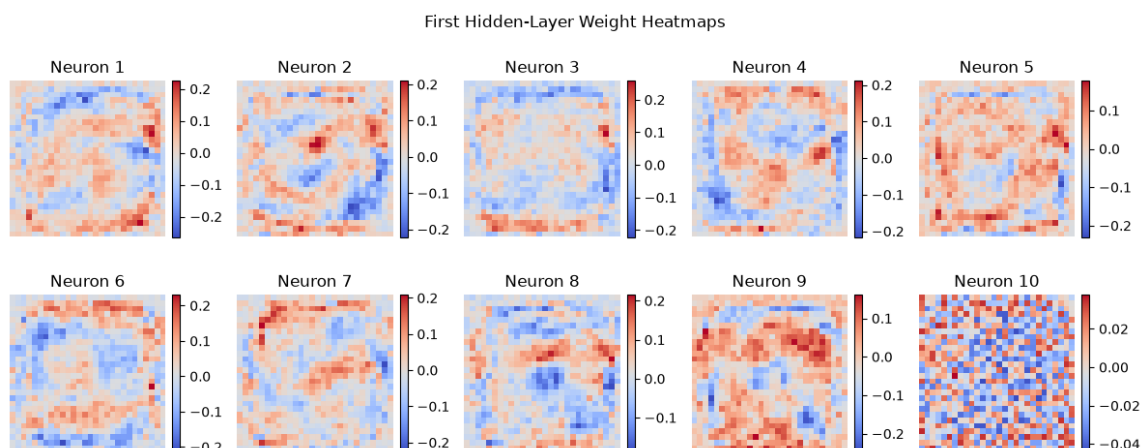


Figure 3. Ten first-hidden-layer neurons visualized as 28 x 28 weight maps.

The weight maps contain structured positive and negative regions rather than appearing as purely random noise. Different neurons emphasize different parts of the input field, indicating that the first hidden layer learns reusable combinations of pixel values. These structures can correspond to stroke-like or edge-like patterns that help distinguish digits.

3.5 Scrambled-MNIST Experiment

Testing whether a fixed permutation of pixels changes FCNN performance.

A single fixed random permutation of the 784 pixel positions was generated and applied to every training and test image. The permutation destroys the visual layout while preserving a consistent mapping from each original pixel position to a new input coordinate.

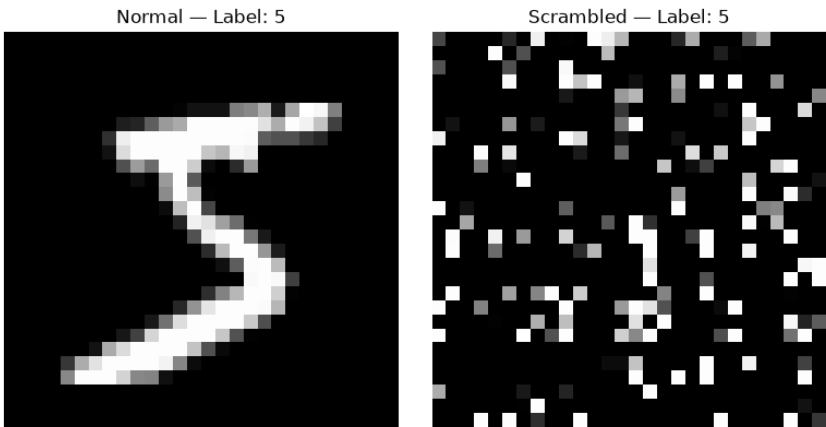


Figure 4. Example of the same digit before and after the fixed pixel permutation.

Dataset	Accuracy
Normal MNIST	97.27%
Scrambled MNIST	97.37%
Difference	−0.10 percentage points

Key result

The accuracy difference is only 0.10 percentage points. The scrambled model is slightly higher in this run, showing that the FCNN can learn the same classification task even after the pixels are consistently rearranged.

A fully connected layer assigns a separate learned weight to every input coordinate and does not inherently encode the idea that nearby pixels should be treated as spatial neighbors. Because the same permutation is applied to every example, the network can learn a new mapping from the permuted coordinates to the digit labels. The experiment demonstrates both the flexibility and the limitation of a standard FCNN: it can exploit coordinate-specific patterns, but it has no built-in two-dimensional spatial prior.

4. Part 3 - Tiny ImageNet: Stress Testing and Robustness

A deeper FCNN is used to expose gradient-flow and optimization issues.

4.1 Problem Definition

Part 3 evaluates an eight-hidden-layer FCNN on a 10-class subset of Tiny ImageNet. Experiment 3.1 compares deep Sigmoid activations against ReLU with Batch Normalization to study vanishing gradients. Experiment 3.2 changes one training factor at a time to measure the effect of Dropout, learning rate, and optimizer.

4.2 Dataset and Architecture

The selected subset contains 10 classes, 5,000 training images, and 500 validation images. Each image is RGB at 64 x 64 pixels, giving 12,288 input features after flattening.

Layer	Units / operation
Input	12,288
Hidden 1	1,024
Hidden 2	512
Hidden 3	256
Hidden 4	128
Hidden 5	128
Hidden 6	64
Hidden 7	64
Hidden 8	32
Output	10 classes

Training setup

Cross-Entropy loss, Adam optimizer, learning rate 0.001, and 5 epochs were used for the gradient-flow comparison. The ablation baseline added Dropout(0.2) to the ReLU + BatchNorm architecture.

4.3 Experiment 3.1 - Vanishing Gradients

Deep Sigmoid versus ReLU + BatchNorm.

Epoch	Sigmoid loss	Sigmoid $\ dW_1\ $	ReLU+BN loss	ReLU+BN $\ dW_1\ $
1	2.3134	4.078e-7	2.1820	1.1384
2	2.3051	7.790e-8	2.0341	0.1354
3	2.3047	6.924e-8	1.9289	0.1965
4	2.3057	8.318e-8	1.8237	0.3136
5	2.3054	1.164e-7	1.7474	0.3020

10^{-7} to 10^{-8}

Sigmoid first-layer gradients

10^{-1} to 10^0

ReLU+BN gradients

2.303

10-class random-guess loss

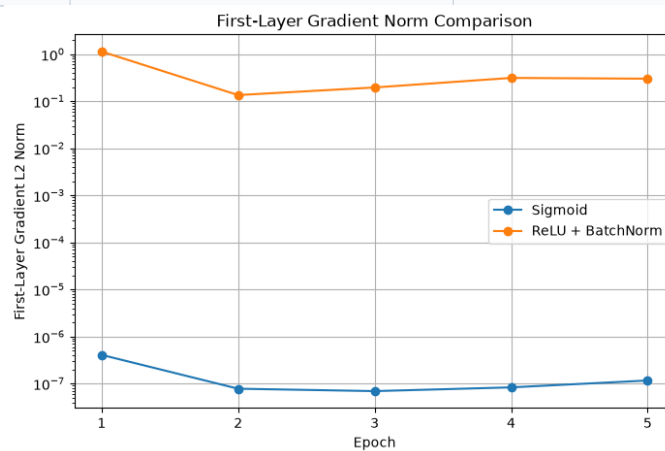


Figure 5. First-layer gradient norm comparison on a logarithmic scale.

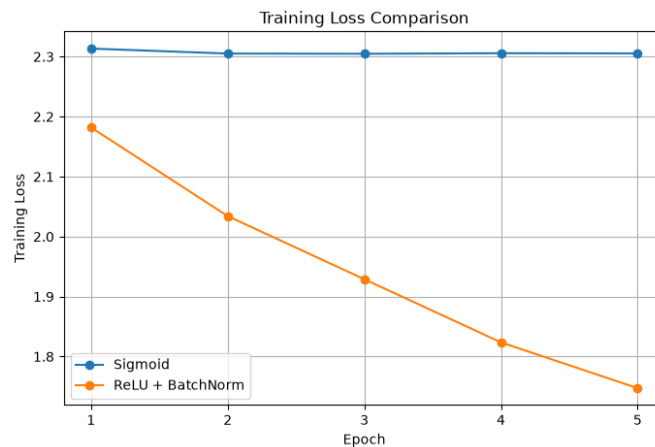


Figure 6. Training loss comparison for the deep networks.

4.4 Interpretation

The Sigmoid network shows severe gradient attenuation. Its first-layer gradient remains around 10^{-7} to 10^{-8} and its loss stays close to $\ln(10)$, approximately 2.303, which is the cross-entropy level associated with near-random predictions across ten classes. In contrast, the ReLU + BatchNorm model retains much larger first-layer gradients and reduces its loss from 2.1820 to 1.7474.

Conclusion

Repeated Sigmoid derivatives can shrink gradients as they propagate through many layers. ReLU avoids the same systematic contraction for active units, while Batch Normalization helps stabilize activation distributions and optimization.

4.5 Experiment 3.2 - Ablation Study

Each experiment changes one factor relative to the same baseline.

The baseline uses ReLU + BatchNorm, Dropout(0.2), Adam, and learning rate 0.001. Each ablation starts from a fresh model and changes one factor while keeping the main settings fixed.

Experiment	Validation accuracy	Change vs baseline
Baseline	29.20%	0.00 pp
No Dropout	31.60%	+2.40 pp
Learning rate x10 (0.01)	28.20%	-1.00 pp
Learning rate /10 (0.0001)	18.60%	-10.60 pp
Adam -> Vanilla SGD	9.80%	-19.40 pp

31.60% best ablation	-19.40 pp largest drop	5 epochs per run
--------------------------------	----------------------------------	----------------------------

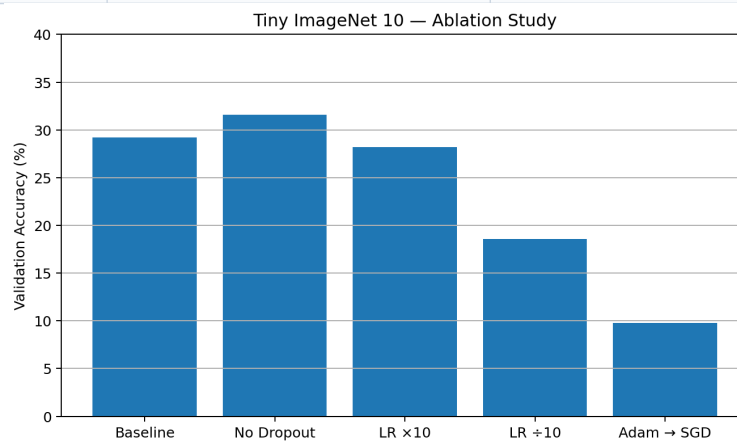


Figure 7. Final validation accuracy across the ablation configurations.

4.6 Ablation Interpretation

Factor	Observed effect	Interpretation
Dropout	+2.40 pp without Dropout	Dropout was not beneficial within the 5-epoch budget.
Learning rate x10	-1.00 pp	The larger step size caused only a small degradation.
Learning rate /10	-10.60 pp	Optimization became substantially slower under the limited training budget.
Adam -> SGD	-19.40 pp	Optimizer choice had the largest measured impact.

Largest effect	Replacing Adam with vanilla SGD reduced validation accuracy from 29.20% to 9.80%. Among the tested factors, optimizer choice had the strongest effect on final validation performance.
-----------------------	--

5. Learned Feature Interpretation

What the completed experiments reveal about learned internal representations.

Scope note

The submitted notebooks do not contain a separate PCA, t-SNE, or UMAP latent-space visualization. Therefore, no unsupported low-dimensional clustering result is claimed in this report.

5.1 What the First Hidden Layer Learns

The MNIST first-layer weight maps provide the most direct completed representation analysis. The neurons learn distributed combinations of input pixels, with different neurons emphasizing different regions and stroke-like structures. This shows that the FCNN learns reusable feature patterns rather than treating every pixel independently at the decision stage.

5.2 What the Scrambling Experiment Reveals

The almost identical normal and scrambled MNIST accuracies, 97.27% and 97.37%, reveal an important property of fully connected layers. The model can learn a coordinate-specific mapping after a fixed permutation; spatial locality is not encoded as an architectural prior in a standard FCNN.

5.3 Representation Takeaway

The learned representation is flexible but not spatially structured by design. The network can discover useful combinations of input coordinates, but the meaning of those coordinates comes from the training data and learned weights rather than from an architectural prior about image locality.

6. Overall Findings and Conclusions

The main lessons from the three experiments.

Finding	Evidence from the experiments
Manual FCNN mechanics can be implemented from first principles	Part 1 implements initialization, forward propagation, BCE, backpropagation, and SGD with NumPy.
Feature scaling improves optimization behavior	Scaled Adult data reached final loss 0.445844 versus 0.565254 for raw features and retained non-zero first-layer gradients.
FCNNs can classify flattened images effectively	The MNIST FCNN reached 97.27% accuracy in the normal-data experiment used for scrambling.
FCNNs have no inherent spatial prior	Scrambled MNIST reached 97.37%, 0.10 percentage points above normal MNIST.
Deep Sigmoid networks suffer gradient attenuation	First-layer Sigmoid gradients stayed around 10^{-7} to 10^{-8} and loss remained near 2.303.
ReLU + BatchNorm improves gradient flow	First-layer gradients remained around 10^{-1} to 10^0 and loss decreased steadily.
Optimizer choice can dominate deep-FCNN performance	Adam -> SGD caused the largest ablation drop, -19.40 percentage points.
Final conclusion	Across the three parts, the experiments demonstrate that FCNN performance depends not only on network capacity but also on input scaling, activation functions, normalization, optimizer choice, and the inductive biases built into the architecture.

6.1 Important Context for Results

The Adult target is imbalanced, so a roughly 76-77% accuracy should not be interpreted as excellent classification performance. The Tiny ImageNet results were obtained with only five epochs, so the ablation study measures performance under the specified training budget rather than the maximum achievable performance of each configuration.

7. Submission Notes

The accompanying notebooks contain the executed implementations and experiments for Parts 1, 2, and 3. This report is the consolidated presentation of those results, with test accuracy and validation accuracy distinguished where appropriate.

End of report